Ticket to Ride
Design Document

Author:
Josh Brian Saputra

Table of Contents

Table of Contents **2**
1 Introduction **4**
2 Updated Class Diagram **4**
2.1 Extensions Overview 4
2.2 Class Diagram 6
2.3 Changes from Sprint 2 6
3 Antipatterns and Refactoring **8**
Antipattern 1: God Class 8
Antipattern 2: Feature Envy / Inappropriate Intimacy 9
Antipattern 3: Anemic Domain Model on Route 9
4 Design Rationales **10**
4.1 New and Updated Classes 10
4.2 Inheritance and Cardinality Decisions 11
4.3 Alternative Designs Considered 12
4.4 Quality Attribute: Modifiability 13
4.5 Design Patterns 14
5 Supplementary Documentation **15**

Document Revision History

Date	Version	Description	Author
10/05/2026	0.1	Created Sprint 3 document structure	Josh Brian Saputra
12/05/2026	0.2	Completed extensions overview and class diagram	Josh Brian Saputra
13/05/2026	0.3	Completed antipatterns and refactoring section	Josh Brian Saputra
14/05/2026	0.4	Completed design rationales	Josh Brian Saputra
30/05/2026	1.0	Finalised report writing, formatting, and consistency checks	Josh Brian Saputra

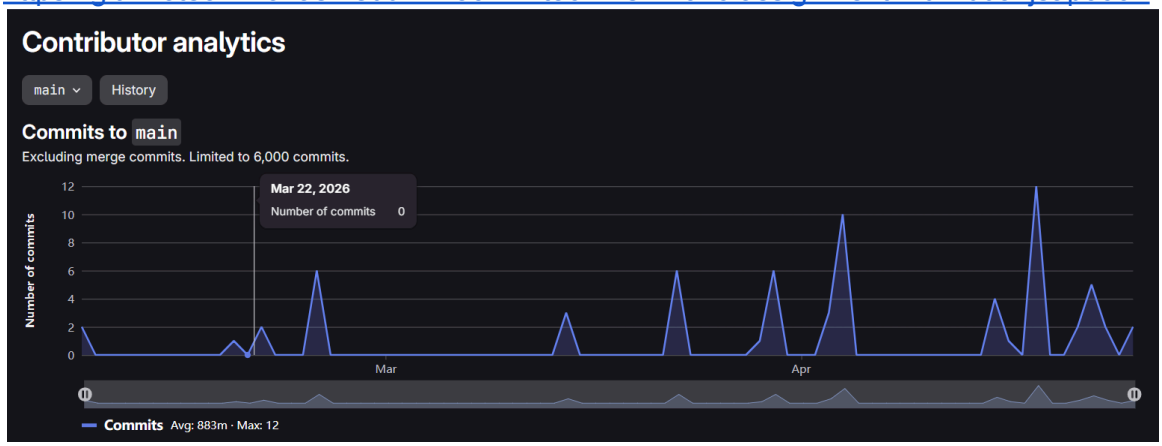
1 Introduction

This document presents the updated software design for the Ticket to Ride system, extended individually during Sprint 3. It builds directly on the Sprint 2 group submission produced by Team DevOops and documents the three extensions implemented, the architectural changes made to support them, and the reasoning behind all significant design decisions.

The document is organised as follows:

- **Updated Class Diagram:** describes the three chosen extensions and illustrates all structural changes from Sprint 2
- **Antipatterns and Refactoring:** identifies three design antipatterns present in the Sprint 2 codebase and explains the refactoring techniques applied
- **Design Rationales:** justifies key design decisions, discusses quality attributes, and evaluates design patterns
- **Supplementary Documentation:** additional notes relevant to the submission

GitLab repository:
https://git.infotech.monash.edu/FIT3077/fit3077-s1-2026/assignment-individual/jsap0007



2 Updated Class Diagram

2.1 Extensions Overview

Three extensions were implemented for Sprint 3:

- **Required: Ferries**
  Ferry routes are special routes requiring one Locomotive card per ferry symbol on the route, with remaining spaces filled by cards of the route's colour. Extra Locomotive cards may substitute colour cards as wilds. If a player lacks sufficient Locomotive cards, any three colour cards may replace each missing Locomotive. Ferry routes are defined in board.json via a ferry_locos integer field and are visually marked on the map with anchor badges. Full ferry arithmetic including the three-for-one substitution is encapsulated in Route.compute_card_slots().
- **Pick-from-list: Dynamic Configuration**
  The game supports multiple maps and two to five players configured at the start of each session. Each map is an external folder containing a board.json (cities

and routes) and a map.png (background image). The setup screen scans a maps/ directory and presents selectable map cards. Destination tickets are no longer hardcoded, instead Board.generate_tickets() uses all-pairs Dijkstra shortest paths to automatically produce an appropriate ticket set from any valid board definition, meaning a new map requires no code changes at all.

- **Self-defined: Detailed Illegal Move Feedback**
  When a player attempts an illegal action, the game provides a precise, contextual explanation rather than a generic error. For example, attempting to claim a ferry route with insufficient locomotives produces a message such as: "Ferry route needs 2 locomotive(s) for the water crossing. You are short 1 locomotive(s). Each can be replaced by 3 Blue cards (3 extra cards total). Total Blue cards required: 5. You have 3. Short by 2 card(s)." This is handled by a dedicated MoveValidator class returning a structured ValidationResult containing both a reason and a hint string. The UI surfaces the reason as a dialog title and the hint as a secondary label in the sidebar.

2.2 Class Diagram

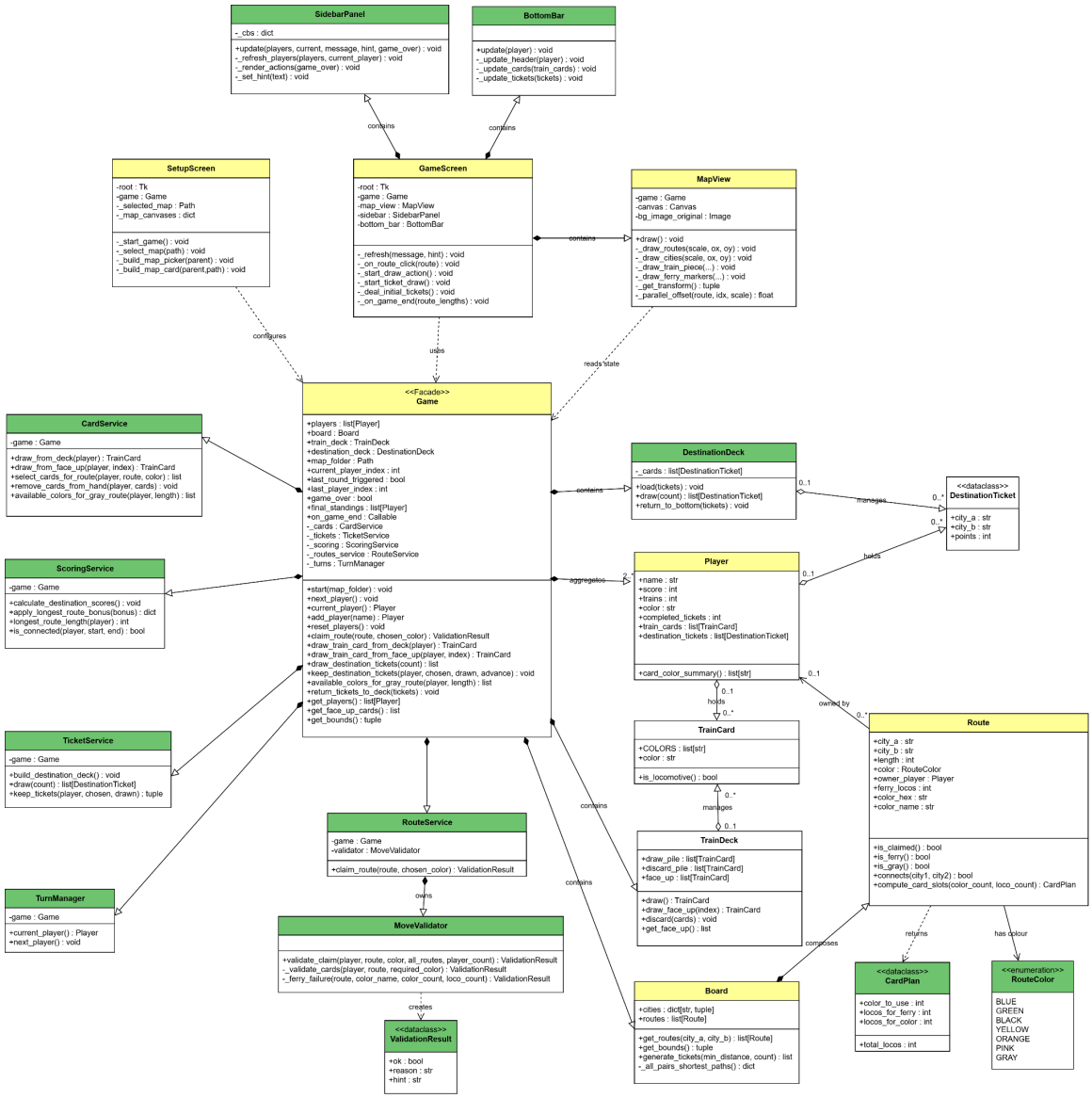


Figure 2.1: Updated Class Diagram

New classes are shown in green. Classes modified from Sprint 2 are shown in yellow. Unchanged classes are shown in white.

2.3 Changes from Sprint 2

The following summarises all structural changes visible in the diagram.

- **Game (modified, now a Facade).** In Sprint 2 Game contained all game logic directly: route claiming, card selection, destination ticket management, scoring, DFS connectivity checks, and turn advancement. In Sprint 3 this logic is extracted into dedicated service classes. Game retains only state fields and thin

delegation methods, acting as a Facade that shields the UI from internal service wiring.

- **CardService (new).** Owns all train card draw and selection logic previously scattered across Game. Provides draw_from_deck(), draw_from_face_up(), select_cards_for_route(), remove_cards_from_hand(), and available_colors_for_gray_route().
- **RouteService (new).** Handles route-claim transactions: calls MoveValidator, delegates card selection to CardService, updates Route.owner_player, deducts trains, awards points, and triggers last-round detection. This was previously all inline in Game.claim_route().
- **ScoringService (new).** Owns end-game scoring: destination ticket evaluation (DFS connectivity), longest route calculation, and the longest-route bonus. The DFS that was private inside Game in Sprint 2 now lives here.
- **TicketService (new).** Owns destination ticket draw and keep logic, including calling Board.generate_tickets() to populate the deck at game start.
- **TurnManager (new).** Owns turn index advancement. Extracted from Game.next_player().
- **MoveValidator (new).** Contains all rule-check logic for claiming a route: ownership, parallel route blocking in small games, train count, and card sufficiency. Returns a ValidationResult dataclass with ok, reason, and hint fields. Card sufficiency is fully delegated to Route.compute_card_slots() so no arithmetic is duplicated.
- **ValidationResult (new dataclass).** Structured return type from MoveValidator.validate_claim(). Having a named return type rather than a raw tuple makes the contract between RouteService, MoveValidator, and the UI explicit.
- **CardPlan (new dataclass).** Returned by Route.compute_card_slots(). Carries color_to_use, locos_for_ferry, and locos_for_color as named fields. Replaces the implicit integer arithmetic that was previously spread across Game.select_cards_for_route() and repeated in validation.
- **Route (modified).** Gained ferry_locos: int, compute_card_slots(), is_ferry(), is_gray(), color_hex, and color_name. color changed from an optional string to a RouteColor enum value, eliminating raw string comparisons throughout the codebase.
- **RouteColor (new enum).** Defines the six named route colours and their hex values. Centralises colour definitions that were previously magic strings.
- **Board (modified).** Now loads from board.json rather than hardcoding cities and routes. Added generate_tickets() and the private _all_pairs_shortest_paths() Dijkstra implementation.
- **DestinationDeck (new).** Encapsulates the destination ticket draw pile: load(), draw(), return_to_bottom(). In Sprint 2 Game managed the destination list directly as a plain Python list.

- **SetupScreen (modified).** Added map discovery (_find_maps()), a visual map card picker, and player colour badges. The start() call now passes the selected map folder path.
- **BottomBar (new).** Separated from the monolithic GameScreen of Sprint 2. Displays the active player's train card hand (grouped by colour) and destination tickets with a scrollable canvas.
- **SidebarPanel (new).** Separated from GameScreen. Shows the player roster with live scores and trains, the turn info/hint labels, and the action buttons.
- **MapView (modified).** Added background image rendering, 3D train piece drawing (shadow/body/highlight layers), ferry anchor badge markers, and the parallel-offset algorithm for double routes.

3 Antipatterns and Refactoring

Antipattern 1: God Class

Where it was found.
Sprint 2's Game class was responsible for every aspect of the game: adding and managing players, initialising the board and decks, drawing and discarding train cards, selecting cards for routes, claiming routes and awarding points, managing the destination ticket deck, DFS connectivity checking, longest-route calculation, end-game scoring, and turn advancement. The class was approximately 300 lines long and contained at least eight distinct areas of responsibility.

Why it is problematic.
A God Class directly violates the Single Responsibility Principle. Any change to one concern for example, updating ferry card arithmetic required modifying the same file as turn management and scoring. This made the class fragile: a bug fix in one area risked introducing regressions in another. It also made testing impossible in practice, since no individual behaviour could be exercised in isolation.

Refactoring applied: Extract Class.
All distinct responsibilities were moved into dedicated service classes: CardService, RouteService, ScoringService, TicketService, and TurnManager. A sixth class, MoveValidator, handles rule validation. Game was retained as a thin Facade that holds shared state and delegates to these services via private references (self._cards, self._routes_service, etc.). Each service receives a reference to Game on construction and reads or writes state through it, keeping state centralised while distributing logic.

Improvement.
Each service class can now be read, modified, and reasoned about independently. Adding ferry support, for example, required changes only to RouteService, CardService, MoveValidator, and Route. Game itself needed no logic changes. Cohesion within each class increased significantly, and inter-class coupling is explicit and controlled through the Facade interface.

Antipattern 2: Feature Envy / Inappropriate Intimacy

Where it was found.

In Sprint 2, game_screen.py (the UI class) directly manipulated the internal structure of Game's destination deck. The UI was aware that destination_deck was a plain Python list, how it was ordered, and how returning tickets worked. Similarly, game_screen.py accessed self.game.train_deck.face_up directly to build the face-up card buttons rather than going through a method on Game.

Why it is problematic.

When a UI class reaches into the internals of a domain class to manipulate data directly, any change to how that data is stored breaks the UI. This is Inappropriate Intimacy: the UI class knows too much about the domain class. It also means business logic (what "returning a ticket" means) leaks into the presentation layer, making both harder to maintain.

Refactoring applied: Move Method / Encapsulate Field.

DestinationDeck was introduced as a dedicated class with a return_to_bottom() method. Game exposes return_tickets_to_deck() and draw_destination_tickets() as its public interface. The UI in Sprint 3 only ever calls methods on Game. It never touches destination_deck or train_deck directly. Similarly, Game.get_face_up_cards() wraps train_deck.get_face_up() so the UI does not depend on TrainDeck at all.

Improvement.

The UI is now decoupled from the storage representation of the deck. If DestinationDeck were changed from a list to a deque or database-backed store, GameScreen would require no changes. The boundary between the presentation layer and the domain layer is now enforced structurally rather than by convention.

Antipattern 3: Anemic Domain Model on Route

Where it was found.

In Sprint 2, Route was essentially a data container with two trivial predicate methods (is_claimed(), connects()). All card-sufficiency logic determining how many of which cards were needed to claim a route lived in Game.select_cards_for_route(). The Route object had no knowledge of how it could be claimed. When the ferry extension was designed, this arithmetic would have needed to be replicated in at least three places: card selection, validation, and the error message builder.

Why it is problematic.

An Anemic Domain Model places behaviour that belongs to an object outside of it. Route knows its length, colour, and cities, it is the natural home for "how many cards of what

type are needed to claim me?" Keeping this logic in Game means it cannot be reused, tested, or extended without modifying Game.

Refactoring applied: Move Method.
The card-requirement logic was moved into Route.compute_card_slots(), which returns a CardPlan dataclass. The ferry arithmetic, including the three-for-one substitution, is entirely encapsulated here. CardService.select_cards_for_route() and MoveValidator._validate_cards() both call route.compute_card_slots() and consume the returned CardPlan. The arithmetic exists in exactly one place.

Improvement.
Adding ferry support required writing the ferry branch of compute_card_slots() once. No other class needed to re-implement or copy this logic. MoveValidator can produce accurate, detailed error messages (including exact shortfall counts) simply by inspecting the inputs and the CardPlan it would have needed without duplicating arithmetic. The domain object is now behaviourally rich and genuinely useful.

4 Design Rationales

4.1 New and Updated Classes

Service classes (CardService, RouteService, ScoringService, TicketService, TurnManager).
Each service was introduced to hold a single well-defined responsibility extracted from the Sprint 2 God Class. The question of whether their responsibilities could have stayed in Game was explicitly considered: technically yes, but doing so would have perpetuated the SRP violation and made the ferry extension significantly harder to implement cleanly. Routing ferry card arithmetic through Game would have required modifying a class already responsible for turn management, scoring, and player management. Separating services means each can be modified or tested independently.

Known trade-off: circular dependency between Game and services.
Each service holds a self.game back-reference while Game holds a reference to each service. This creates a circular dependency, a known design antipattern where neither Game nor any service can be instantiated or tested without the other. The trade-off was accepted because the alternative (injecting only the specific state each service needs) would have required either many constructor parameters or a dedicated state container class, both of which add complexity that is not justified at this scale. The circular dependency is contained. All cycles go through Game as the single shared state owner, so the coupling is at least centralised and explicit rather than spread across unrelated classes.

MoveValidator and ValidationResult.
MoveValidator could have been merged into RouteService, since RouteService is the only current caller. It was kept separate for two reasons. First, validation logic is

conceptually distinct from transaction logic: validating that a claim is legal is a different concern from executing the claim. Second, the detailed error messages required for the self-defined extension are substantially more complex than a pass/fail check, and co-locating this complexity with the route transaction would have made RouteService harder to read. ValidationResult as a named dataclass was chosen over a tuple (bool, str, str) because named fields make the contract explicit at every call site and eliminate positional confusion.

DestinationDeck.
In Sprint 2 the destination ticket pile was a plain list on Game. It was extracted into DestinationDeck for the same reason TrainDeck exists as a class: deck behaviour (draw, return to bottom, shuffle on load) is a coherent responsibility that should not live on the game controller.

CardPlan.
This dataclass exists purely to give a name to the return value of compute_card_slots(). Before its introduction, any caller that needed to know "how many colour cards and how many locomotives" would have had to re-derive the values or interpret a positional tuple. CardPlan makes the contract between Route, CardService, and MoveValidator self-documenting.

RouteColor enum.
Sprint 2 used raw lowercase strings ("blue", "green", None for gray) for route colours throughout the codebase. This meant colour comparisons were scattered string operations with no compiler or interpreter assistance if a string was misspelled. RouteColor centralises colour definitions, associates each colour with its hex value (removing a separate lookup dictionary), and makes illegal colour values impossible to construct.

BottomBar and SidebarPanel.
Sprint 2's GameScreen built and managed both the sidebar and the bottom panel inline. These were separated into their own classes because each has a clearly distinct rendering concern. BottomBar only knows about the current player's hand and tickets. SidebarPanel only knows about the player roster, turn messages, and action buttons. GameScreen now acts as a layout coordinator, delegating refresh calls to each panel. This mirrors the Extract Class refactoring applied to Game.

4.2 Inheritance and Cardinality Decisions

Inheritance.
The most significant inheritance decision of the rejected FerryRoute subclass is discussed in full detail under Alternative Designs (section 4.3). In short, the difference between ferry and standard routes is a single integer field and a branching computation, which does not justify a new type in the hierarchy.

Similarly, although TrainCard and DestinationTicket both represent card-like objects, no shared parent Card class was introduced. Their behaviours are entirely different. One is used for route claiming, the other for scoring objectives and forcing them under a common hierarchy would have produced an abstraction with no meaningful shared interface.

Cardinality.
The cardinalities in the diagram were chosen to reflect how objects interact at runtime, not just structurally.

- **Game → Player (1 to 2..*):** a game requires at least two players to be valid. The lower bound of two is enforced in Game.start() which raises ValueError if fewer are registered. There is exactly one Game instance managing all players.
- **Board → Route (1 to 1..*):** a board must contain at least one route to constitute a valid game map. Routes are defined inside board.json and do not exist independently of the board that loads them. This is composition with a minimum cardinality of one.
- **Player → TrainCard (1 to 0..*):** a player can hold zero cards after spending them to claim a route, so the lower bound is zero. Cards are transferred between the deck and the player during play. This is aggregation because the cards exist independently in TrainDeck before and after being held by a player.
- **Player → DestinationTicket (1 to 0..*):** players begin with zero tickets before the initial deal. Tickets are drawn from DestinationDeck and may be returned to it, so again aggregation with a lower bound of zero.
- **Route → Player (0..* to 0..1):** a route is either unclaimed (associated with zero players) or owned by exactly one player at any point. A player can own zero or more routes across the board. Route ownership is a temporary game state. The route exists regardless of whether it has been claimed.
- **DestinationDeck → DestinationTicket (1 to 0..*):** the deck may be empty mid-game if all tickets have been drawn and none returned. The lower bound of zero is correct.

4.3 Alternative Designs Considered

Alternative for ferry logic: subclass of Route.
An early consideration was to introduce a FerryRoute subclass of Route that overrode the card-requirement calculation. This was rejected because:

1. ferry routes differ from standard routes only in one integer field (ferry_locos) and a branching computation. The difference does not justify a new class in the inheritance hierarchy.
2. Python dataclasses and frozen instances interact poorly with subclassing.
3. The is_ferry() predicate plus a single branching method in compute_card_slots() is simpler and equally readable. Introducing inheritance here would be an example of unnecessary abstraction.

Alternative for map loading: database or registry class.
A MapRegistry class was considered to own the scanning and loading of maps from the file system. This was not implemented because the logic is simple enough (scan a directory for folders containing two required files) that adding a registry class would introduce indirection without benefit. The scanning is a one-time setup step performed by SetupScreen, a dedicated class would only be warranted if map loading needed to be done lazily, incrementally, or from multiple sources.

Alternative for destination ticket generation: hardcoded tickets per map.
The Sprint 2 approach was a hardcoded list of 20 tickets in Game. An alternative for Sprint 3 would have been to include a tickets.json alongside each board.json. This was rejected in favour of the Dijkstra-based generator because:

1. It requires no per-map authoring work. Adding a city to board.json automatically produces appropriate tickets on the next game start.
2. The generated point values are proportional to actual path distance, making the tickets inherently balanced.
3. It avoids the risk of hardcoded tickets referencing cities that were later renamed or removed from the board.

4.4 Quality Attribute: Modifiability

Scenario.
A developer adds a new rule variant. For example, a tunnel route type requiring an additional random card draw to the system. The change should be isolated to the fewest possible classes, take less than a few hours to implement, and not require modifications to UI classes.

Analysis of Sprint 2.
In the Sprint 2 design this change would have required modifying Game.select_cards_for_route(), Game.claim_route(), and game_screen.py which are three classes with different responsibilities. Because card arithmetic was not encapsulated in Route itself, any new route variant would have needed new branching logic in the game controller and potentially in the UI as well.

Tactic applied: Increase Semantic Coherence (Encapsulate).
The primary tactic used to improve modifiability is encapsulation of variability within the domain object. Route.compute_card_slots() is the single point of extension for new route mechanics. Adding a tunnel variant requires:

1. Adding a tunnel: bool field to Route.
2. Adding a tunnel branch in compute_card_slots().
3. Updating board.json for the relevant routes.

No UI class and no service class beyond RouteService would need modification.

Trade-off.

The service architecture adds indirection. A route claim now flows through Game.claim_route() → RouteService.claim_route() → MoveValidator.validate_claim() → CardService.select_cards_for_route(). This increases the number of files a developer must navigate to follow a single action. The trade-off is accepted because of the modifiability benefit, particularly for the ferry extension, which required no changes to Game itself which validates the architectural choice. For a project of this scale the indirection remains tractable.

4.5 Design Patterns

Facade (applied).

Game is a Facade over the five service classes. The UI layer (GameScreen, BottomBar, SidebarPanel) interacts exclusively with Game and is shielded from the internal service topology. This pattern was appropriate because:

1. It preserves backward compatibility with the UI from Sprint 2. The same method names on Game are available, only their implementations changed
2. It allows the internal service structure to be restructured without touching any UI class
3. It provides a single, comprehensible entry point for understanding what the game can do.

Patterns considered but not applied:

Command Pattern.

The Command pattern encapsulates each action as an object, enabling undo/redo and action queuing. It was considered because the undo extension was on the pick-from-list. Since the dynamic configuration extension was chosen instead, Command was not warranted.

Observer Pattern.

Observer would allow the UI to register as a listener on Game and refresh automatically when game state changes, rather than requiring explicit _refresh() calls after each action. This was considered and rejected for the same reason as Sprint 2: the current notification surface is small (one callback, on_game_end) and explicit refresh calls at known action completion points are easy to follow. A full Observer implementation would add observable/listener infrastructure for marginal benefit at this scale.

State Pattern.

A State pattern could model the turn phases (main menu, drawing cards, drawing tickets) as discrete state objects, replacing the _action_mode string that existed in Sprint 2's GameScreen. In Sprint 3 this was implicitly resolved differently: each action (_start_draw_action, _start_ticket_draw, _on_route_click) is self-contained and modal dialogs are used for multi-step interactions, eliminating the need for explicit mode tracking on the screen class. State would be more appropriate if the turn flow became significantly more complex.

5 Supplementary Documentation

The map images used in the user interface were generated using ChatGPT. No other generative AI was used to produce artwork or assets.

Two defects from Sprint 2 were corrected during Sprint 3 development. First, the Sprint 2 implementation started each player with 45 trains and dealt 4 initial train cards. The London map variant uses 17 trains and 2 initial cards per player. This was corrected alongside the refactoring work. Second, the parallel route blocking rule in Sprint 2 checked len(self.players) <= 3 but the correct London rule restricts parallel routes in 2-player games only. The Sprint 3 MoveValidator corrects this check to player_count == 2.

The class diagram was designed before implementation and used to guide the service extraction. Minor deviations between the diagram and final implementation exist in MapView (several private rendering helpers omitted from the diagram for clarity) and in BottomBar (internal tkinter widget references omitted). These are purely rendering utilities with no bearing on game logic or architecture.